Digital Campus Library Reservation Gateway — UML Component Diagram

Architectural style: Microservices | SE Lab 3 — Team 08 | Kishan K Shenoy, PES1UG24AM138

CLIENT TIER

«component»
Member Portal
Student Member — web / mobile UI

«component»
Librarian Console
Head Librarian — catalog admin UI

IMemberAPI
REST/HTTPS

ILibrarianAPI
REST/HTTPS

EDGE / ACCESS TIER

«component»
API Gateway
TLS termination · JWT auth · RBAC routing

ICatalogAPI
REST/JSON

IHoldAPI
REST/JSON

ICirculationAPI
REST/JSON

IFineAPI
REST/JSON

BUSINESS SERVICES TIER

«component»
Catalog Service
ISBN search, add / update /
deactivate titles (FR-001, FR-004)

«component»
Reservation Service
Holds, FIFO queue,
48h release timer (FR-001, FR-002)

«component»
Circulation Service
Check-out / check-in,
renewals (FR-002, FR-005)

«component»
Fine Service
Daily overdue-fine
calculation (FR-003)

«component»
Notification Service
Email / SMS dispatch
within 5 min (FR-002)

ICatalogLookup (REST)

IHoldQueue (REST)

IFinePosting (REST)

IHoldNotify / ILoanNotify «async / AMQP» : sendHoldReady, sendHoldLapsed, sendReceipt

DATA TIER & EXTERNAL SYSTEMS

ICatalogData
SQL/JDBC

IHoldData
SQL/JDBC

ILoanData
SQL/JDBC

IFineData
SQL/JDBC

ISendMessage
SMTP/SMS API

«database»
Catalog DB
ISBN repository

«database»
Reservation DB
holds + queues

«database»
Loan DB
loans + due dates

«database»
Fine DB
fine transactions

«external system»
Email / SMS Provider
external SMTP / SMS API

Provided interface

Required interface

Sync assembly connector (REST/SQL)

Async dependency (AMQP queue)

Lab 3: Component Modelling & Architectural Pattern Selection

Digital Campus Library Reservation Gateway | Team 08 | Problem Statement #08

Kishan K Shenoy — PES1UG24AM138

Architecture Selection: "We chose Microservices Architecture for the Digital Campus Library Reservation Gateway System." The system is split into five independently deployable services — Catalog, Reservation, Circulation, Fine, and Notification — reached through a single API Gateway, each owning its own database (see the accompanying component diagram).

Style Comparison (Step 2)

Style	Strengths for this scenario	Weaknesses for this scenario	Fit
Layered	Straightforward to build and reason about; works well for a small team and a first iteration.	Ships as one deployable unit, so the catalog search path cannot be scaled on its own, and a single node failure can take the whole system down (conflicts with NFR-002).	Medium
Client-Server	Centralised control and a simple, consistent data model.	The server is a single point of failure and a scaling bottleneck, directly at odds with the 99.9% uptime and 5-minute recovery target.	Low
Microservices	Each function scales and fails independently; services can be deployed and owned separately.	Adds network calls and eventual consistency between services (addressed in the trade-off section below).	High

Two Reasons for the Choice

1. Fault isolation keeps the core reservation flow alive (NFR-002). The gateway is required to stay up 99.9% of the semester and recover from a single-node failure within 5 minutes. Because each service is deployed and scaled independently, a crash in the Fine Service while it posts overdue charges, or a slow restart of the Notification Service, does not stop a student from searching the catalog or placing a hold — the services most students touch every day. A layered or client-server build has no such boundary: one failing node brings the whole application down.

2. The workload is uneven across features (NFR-001, FR-002). Catalog search has to stay under 150 ms across 50,000 titles and spikes hard during enrolment and exam weeks, while fine calculation and hold-expiry timers are light, scheduled background work. Splitting these into separate services means the Catalog Service can be given extra capacity on its own, without over-provisioning the lighter Fine or Notification services just to keep up with search traffic.

Security Advantage

All requests pass through the API Gateway, which terminates TLS, validates the member's JWT, and enforces role-based access before anything reaches a service — so only the Head Librarian role can call the catalog's add / update / deactivate operations (FR-004), and a Student Member can only ever see their own holds and fines. Because each service owns its own database, a vulnerability or compromised credential in one service (say, Notification) cannot be used to read another service's data, such as fine transactions or loan records.

Performance Benefit

The Catalog Service can be horizontally scaled and fitted with its own read cache and search index, independent of every other service — this is what lets the 150 ms search target (NFR-001) hold up under peak load without over-provisioning the rest of the system. Likewise, the Notification Service dispatches email/SMS asynchronously over a message queue: a book check-in returns to the member instantly, while the actual notification is still delivered within the 5-minute window required by FR-002, even if the external email/SMS provider is briefly slow.

Trade-off and Mitigation

The cost of this choice is extra network calls and eventual consistency — for instance, a hold can be promoted in the Reservation Service a short moment before the member's notification is actually delivered. This is accepted as a reasonable trade-off: inter-service calls use short timeouts with retries, and the Reservation Service remains the single source of truth for the hold queue, so a delayed notification never changes FIFO order or lets two members be granted the same copy.

Component Diagram: see Lab3_Component_Diagram.png — 4 tiers (Client, Edge, Business Services, Data), 10 components, provided/required interfaces for all synchronous (REST/SQL) and asynchronous (AMQP) connections.